Contents

Future Ideas

Deferred features and improvements that are worth revisiting but not currently prioritized. **Open and partly-shipped items only** — entries that have shipped or been rendered obsolete live in the companion doc **`Future_Ideas_Completed.md`**.

Known Critical Bugs

Warm-anon `/play` tReady — session-bootstrap chatter is the new bottleneck (filed 2026-05-16)

Context: after the PlayVsBot start-flow collapse landed on v5.7 and the `/api/session` shared-singleton dedup landed on v5.8 (both shipped — see `Future_Ideas_Completed.md`), the tReady p50 stayed at **~842 ms desktop / 847 ms mobile-throttled** on prod warm-anon `/play`. The structural collapse worked but the wall-clock floor didn't move — the cost is no longer chain-shaped, it's auth/JS-boot-shaped.

Where the floor lives now: the start-flow chain is structurally collapsed to a single `POST /api/v1/play/bot`. The remaining ~840 ms is dominated by JS chunk parse + React boot + the single `/play/bot` round-trip itself + auth/session resolution before child routes render. None of these are removable by another small structural tweak.

Open follow-ups (ordered cheapest-first):

- **Duplicate `/events/stream` open — needs instrumentation first.** The prod waterfall shows two `/events/stream` GETs but `AppLayout.jsx:177–183`'s `reopenSharedStream()` already has an identity-change guard that should prevent firing on cold-mount anon → anon. Most likely culprit is EventSource auto-reconnect after a server-side session race (Better Auth identity probe mid-bootstrap), but without prod instrumentation to confirm I can't pick the right fix. The second open is also not on the perf-ready critical path (Cell 1 visible fires from the synchronous `applyCreateResultHvb` *before* the second open finishes), so the wall-clock cost is closer to **server-side resource leak** than user-perceived latency. Deferred until either a) we see it gating tReady in a future measurement, or b) we add SSE-lifecycle telemetry. **Files involved (when picked up):** `landing/src/lib/useEventStream.js`, `landing/src/lib/rtSession.js`, `backend/src/routes/events.js`.

Larger items (deferred — gated on Hook conversion being a priority sprint focus):

- **Render `/play` before auth resolves** — AppLayout currently awaits `/api/session` before rendering child routes. Render-then-hydrate pattern: render assuming anon, hydrate to authed UI when the session lands. `/play?action=vs-community-bot` is fully anon-OK so this is safe. **~1 day. ~100–200 ms saved.**
- **Inline the opening board into the HTML** — SSR the `/play?action=vs-community-bot` route with a static empty-board placeholder. The `/play/bot` POST fires from a script tag in the HTML head, before the React bundle finishes loading. Removes the JS-boot-then-effect-then-POST chain. **~2 days. ~200–400 ms saved.**
- **Edge-route `/play/bot`** — Cloudflare Worker or Fly multi-region for this one endpoint. Costs a Tokyo user 200+ ms RTT to a single US-East Fly machine today. Needs DB read-replicas at edge or Redis-cache write-through for bot resolution. **~1 week. ~200–500 ms saved, geography-dependent.**

Regression coverage in place: `perf/perf-playvsbot.js` + `perf/perf-playvsbot-mobile.js` (tReady p50/p95 + structural-collapse asserts: 1 `/play/bot` POST, 0 `/rt/tables` POSTs, 0 `/api/token` GETs, 0 `/bots?gameId=` GETs).

Status snapshot (last reviewed 2026-05-16)

Only open and partly-shipped items shown. For shipped/obsolete entries see `Future_Ideas_Completed.md`.

Item	Status
Real-Time Presence — `user:away` refinement	□ Open — heartbeat-based core is shipped (see Completed doc); explicit away-vs-closed distinction still open
Multi-Game Bots (Phase 3.8)	□ Partly shipped — 8 of 25 items landed on dev during V1 QA (additive parts of 3.8.1 + 3.8.2 + tests); remaining UI work deferred
Backend Logs in Admin Log Viewer (pino → DB)	□ Frontend half + viewer UI shipped; backend pino transport still open
Help System — Sprint 5+ post-launch enhancements	□ Open — Sprint 4 closed v1 scope (see Completed doc); Sprint 5 candidates: reranker v2, embedding upgrade, streaming filter, gap-filling, etc.
Guide as Navigation (Command Palette)	□ Open
Tier 2/3 transport instrumentation	□ Partly done — 3 counters live; remaining items in `doc/Observability_Plan.md`
Tournament admin UX overhaul — *post-launch follow-ups*	□ Open — main sprint shipped (see Completed doc); residual polish open
`table.released` per-reason soak monitor	□ Open (post-prod-launch — needs real traffic to be meaningful)
Redis-backed `sseSessions` registry	□ Open (defer until non-Fly hosting is on the table; Fly-Replay covers single-cloud)
Research Log for users — in-platform training journal	□ Open
Warm-anon `/play` tReady — larger items	□ Open (see entry above)

Migration-sensitivity audit (2026-04-23)

Which of the items above actually benefit from shipping *before* prod has real users? Only schema/data-migration costs scale with user volume; UX features cost the same at 0 users or 10,000.

Item	Migration-sensitive?	Verdict
Multi-Game Bots (Phase 3.8)	No — schema already anticipates it (`BotSkill (botId, gameId)` unique from Phase 1.7)	Do via 3.8 on the normal track
Backend Logs → DB	No — just starts writing more rows	Ship anytime
Command Palette	No — UX feature	Ship anytime
Tier 2/3 instrumentation	No — counters, not schema	Ship anytime

The schema-migration sensitive items (Recurring tournaments template/occurrence split, plus the smaller batch folded into Phase 3.7a — bot displayName uniqueness, public profile URL structure, OAuth prod redirects, seeded built-in bot polish) all shipped pre-launch. Nothing in the current backlog is migration-sensitive.

Multi-Game Bots

What: Allow a single bot to have trained models for multiple games (e.g., XO and chess). Today a bot is effectively XO-only — it holds one model. A multi-game bot would hold one model per supported game and compete on each game's leaderboard independently.

Why deferred: Requires a second game to exist on the platform first. The groundwork is already in place — the credits system is game-agnostic (`appId` field), the `Game` table has an `appId` column, and the Credits Plan explicitly notes "a bot can hold one model per supported game." Bot slot limits already govern agent count, not model count.

What it would take: - Schema: `BotModel` table keyed by `(botId, appId)` to hold per-game weights and ELO separately from the bot's top-level record. - Training UI: game selector when starting a training session in the Gym. - Leaderboard: per-game filtering so a bot's XO ELO and chess ELO are tracked independently. - Community bot matchmaking: players select a game, and only bots with a model for that game appear.

Complexity: Medium-to-large. Straightforward once a second game is added; no point building it for a single game.

Research Log for users — in-platform training journal

What: A persistent training journal owned by each user that automatically captures every training session's hyperparameters, results, and per-session notes. Sits alongside the Gym → Sessions tab; surfaces in Profile and (eventually) in the Guide's What's Next recommendations.

Today the corpus (onboarding-after-the-journey.md, gym-sessions-tab.md) explicitly recommends users keep a research log **outside the platform** — a markdown file or notebook of "what I tried, what worked, what didn't." That advice is the right pattern for serious bot training, but pushing the burden onto the user externally guarantees the data never accrues into anything the platform can learn from. Bringing it in-platform produces three compounding wins.

Why:

1. **User memory.** Sessions tab already records every training run; users still can't remember why they picked a given learning rate three weeks ago. Notes attached to each session row close that gap with zero extra workflow — type a sentence, the platform remembers.
2. **Training-data goldmine.** Per-session notes plus their hyperparameters plus the resulting benchmark/ELO are exactly the shape of data the reranker and (eventually) fine-tuning will love. A user writes "this run plateaued because epsilon decay too aggressive"; six months later that's a labelled example for a "training-advice" assistant.
3. **Onboarding accelerator.** New users see what experienced users wrote about similar runs (privacy-gated; opt-in shared notes only). Shortens the "how do I think about hyperparameters" curve from months to days.

What it would take:

- **Schema:** `TrainingSessionNote` table keyed by `(userId, sessionId)`. Fields: `note` (text, ~2 KB cap), `tags` (text[]), `outcome` (enum: success / plateau / regression / inconclusive), `parentNoteId` (nullable — chain follow-ups), `sharedWithCommunity` (bool, default false), timestamps. A separate `ResearchLogEntry` table for ad-hoc entries not tied to a session (planning, retrospectives).
- **Auto-captured fields:** every training session already stores algorithm, hyperparameters (lr, gamma, epsilon, episodes, etc.), pre/post benchmark, ELO delta. The note layer attaches structured commentary to that existing data — nothing duplicated.
- **UI surface (3 places):**
  - **Gym → Sessions tab:** inline "Add note" affordance on each row. Expanding shows prior notes plus a small textarea + outcome dropdown.
  - **Profile → Training journal tab:** all notes chronologically, filterable by algorithm/outcome/tag. Markdown supported. Export to .md for backup.
  - **Help System integration:** when the user asks the Guide a training question, the retrieval pipeline can surface their own past notes (and, post-opt-in, related public notes from other users) as ranked chunks alongside the corpus. Closes the loop on point #2.
- **Privacy:** notes are private by default. Sharing is opt-in per-note; a "publish" toggle scrubs `userId` and adds the note to a public training-notes corpus that the Help System indexes.
- **Streak / habit nudges:** the Guide's What's Next surface can prompt "you've trained 5 times this week — add a note about what you tried" to drive accrual. Discovery reward eligible.

Complexity: Medium. Schema and auto-capture are straightforward (Sessions tab already has the data). UI is 2–3 dev days for the basic version. The cross-cutting Help System integration (point 2) is what makes this transformative; that's another ~1 week and depends on the reranker work currently scoped for Sprint 5+.

Sequencing dependency: ships best after Sprint 3 of the Help System (so user-private notes can be a retrieval source via the same FTS+pgvector path) and after the §2.5 rate limiter (so a "publish your note as community context" action can be safely opt-in without abuse vectors). The auto-capture-only version (notes attached to sessions, no Help integration) is a clean v1 deliverable any time after Sprint 2.

Backend Logs in Admin Log Viewer

> **Status update (2026-04-29):** the frontend half landed — `landing/src/lib/frontendLogger.js` batches errors / warnings to `POST /api/v1/logs` and the admin Log Viewer at `/admin/logs` populates with `source: frontend` rows. `setLogUserId` is wired through AppLayout so user context is captured. Backend pino → DB is the remaining piece; the rest of this entry covers what's left.

What: Route backend (pino) logs into the database so the admin Log Viewer shows all four sources — `api`, `realtime`, and `ai` — alongside the frontend rows already flowing in. Currently pino writes to stdout (visible in the Fly.io log stream) but never reaches the `logs` table.

Why deferred: stdout logs are accessible via Fly.io / `docker compose logs` for now. The viewer is already useful for frontend errors.

Wiring pino to the DB adds write pressure on every request.

What it would take: - **Pino DB transport:** a custom pino transport (or `pino-transport` wrapper) that batches log entries and inserts them into the `logs` table, respecting the existing `pruneIfNeeded` limit. Use `source: 'api'`, `'realtime'`, or `'ai'` depending on origin. - **Log level threshold:** only write INFO and above from the backend to avoid flooding the table with debug noise. DEBUG can remain stdout-only. - **Live tail:** backend log entries flow through the existing `appendToStream('admin:logs:entry', ...)` path automatically once they're written via the same POST handler the frontend logger uses (or via a direct stream emit from the transport).

Complexity: Small-to-medium (~half a day). The DB schema, ingestion endpoint, pruning, frontend logger, and live-tail SSE are all in place — the missing piece is just the pino → DB bridge.

Help System — post-launch enhancements (Sprint 5)

Status: Sprint 4 closed all v1 scope of the Learnable Help System (corpus + retrieval + admin editor + OpenAI-backed `/help/ask` + Guide drawer UI + feedback + browse pages + curation queue + metrics dashboard) — see the entry in `Future_Ideas_Completed.md`. The items below were originally tracked as "Sprint 5+" in `archive/Help_System_Sprint_Tracker.md` and moved here so the tracker stops at v1 scope. Most are data-driven — they want a few weeks of real prod traffic before they pay off.

Source of truth for the v1 build: `archive/Help_System_Sprint_Tracker.md` (closed sprints 1–4). For shipping new data, edit corpus docs via the admin editor at `/admin/help` (HELP_ADMIN gated). Gap-filling candidates surface on the new `/admin/help/metrics` dashboard under "Recent no-source queries".

Triage order (in roughly the order they'd pay off once we have prod data):

- **Corpus gap-filling from real traffic.** Mine `HelpQuery` (especially the `topNoSourceQueries` rollup from §4.3) for unanswered questions and write gap-filling docs. The "+ Doc" affordances on the curation page + metrics dashboard already seed the editor with the question text — this is editorial work, not new code. Wait until the curation queue has ~2–4 weeks of prod feedback before the first pass.
- **Reranker (v2).** Train a small ranker over (query, chunk, signal) triples once ~1k feedback rows are available. Deploy as a `HelpReranker` model behind a `SystemConfig` flag so we can A/B against the current hybrid (FTS + cosine) retrieval.
- **Embedding upgrade.** If `text-embedding-3-small@384` quality plateaus, evaluate BGE-small (same 384 dim — drop-in) or a larger-dim model (requires a `help_chunks.embedding` column-type migration + full reindex). The auto-reindex-on-model-change path from Sprint 2 §2.1 already handles drop-in swaps; the dim change is the only real lift.
- **Streaming content filter.** Today's filter runs post-stream — if the model emits a banned term mid-answer the UI sees it briefly before the replacement lands. Move filtering to the token-buffer level so banned tokens never reach the wire. Trade-off: filter cost per chunk vs. per response.
- **LoRA fine-tuning (v3).** Once high-quality Q&A pairs accumulate (target ~5k HELPFUL-graded pairs), fine-tune a small model on the best of them as a vendor-independent fallback. Complementary to the reranker, not a replacement.
- **Voice input.** Speech-to-text on the Guide chat input. Cheap once the streaming UI is stable; relevant for mobile.
- **Cross-session help history.** Profile-page view of past Q&A so users can revisit a previous answer. Schema is already there (`HelpQuery.userId` + answers); this is purely UI.
- **Per-IP rate limit.** Add an IP-keyed limiter alongside the existing per-user limiter from Sprint 2 §2.5, gated on detecting shared-account abuse in the wild. Until that signal appears, the per-user limiter is sufficient.
- **Auto-redact PII in question text.** Regex scrubber for emails / phone numbers / addresses in the user's question before it goes to OpenAI. Belt-and-suspenders: the existing disclosure ("Don't include personal details") covers the policy side; this would cover the slip.

Guide as Navigation System (Command Palette Evolution)

What: Add a ⌘K command palette — a keyboard-invokable search overlay (Spotlight / Linear-style) that lets users jump anywhere in the app by typing rather than clicking through the nav.

Current navigation structure: - **Desktop:** a top header bar with Play, Gym, Puzzles, Rankings as primary links, plus Stats / Profile / About in-line. Admin links appear for admin users. - **Mobile:** a fixed bottom tab bar (Play, Gym, Ranks, Stats, Profile) plus a hamburger menu that expands the full link list including Settings, FAQ, and About. - **Guide button:** a pulsing "Guide" button sits next to the logo in the header and opens the Getting Started modal, whose cards navigate directly to destinations via `target="_top"` links. Users can hide this button in Settings.

The nav works fine but requires knowing where things live. There's no way to reach a page by typing its name, and no single surface that lists every destination at once.

How the palette would work: - Press □K (Ctrl+K on Windows) from anywhere to open a centered overlay with a search input and a list of destinations. - The list pre-populates with the same links in `MENU_LINKS` — Play, Gym, Puzzles, Rankings, Stats, Profile, About, FAQ, Settings — plus admin links when applicable. - Typing filters the list instantly. Enter or clicking an item navigates and closes the palette. Escape closes without navigating. - A small □K hint badge in the header (next to the Guide button) would make it discoverable.

Relationship to the guide: The guide is visual and onboarding-oriented — it shows the journey from new user to competitor. The palette is speed-oriented for returning users who already know what they want. They serve different moments and can coexist.

Why deferred: The existing nav covers current usage. The palette pays off most when users are frequent enough to remember keyboard shortcuts.

Complexity: Medium (~2 days). Purely frontend — a new React component with a `keydown` listener at the app root, no backend changes needed.

Tier 2/3 transport — instrumentation (partly done)

The first three items (SSE client count, presence-store size, XREAD-loop heartbeat) are wired in `resourceCounters.js`. The remaining items below are tracked more fully in **`doc/Observability_Plan.md`** (SSE broker peak/age, Redis stream XLEN + consumer lag, Web Push delivery metrics). Treat this entry as the short form; work from the Observability Plan when picking up an observability sprint.

Open nice-to-haves — wire them in once real traffic lands or when push starts behaving oddly:

- **Push subscriptions count** — snapshot `db.pushSubscription.count()` to see how many device endpoints we're pushing to. Also useful for sizing UI in the admin health dashboard.
- **Push send metrics** — expose counters from `pushService`: `pushSent`, `pushFailed` (transient, non-404/410), `pushPurged` (dead endpoints). Surfaces success-rate and catches a VAPID misconfiguration quickly.
- **Redis Stream length** — `XLEN events:tier2:stream`. Bounded by MAXLEN=5000 so it'll always cap there, but tracking the value confirms trimming is working and gives a rough "events per minute" signal when cross-referenced with snapshot timestamps.

Low priority, mentioned for completeness:

- **`/api/v1/presence/heartbeat` QPS** — normal load is `(online users) / 15s`. A sudden 10× spike indicates a client-side retry-loop bug.
- **Dispatch → push fan-out counter** — how often `notificationBus.dispatch` actually lands a push vs skips because SSE was online. Useful for tuning which event types should have `push: true` in the REGISTRY.

Effort: each item is ~5–10 LOC in `resourceCounters.js` plus a small `export function` in the owning module (`pushService.js`, `tournament/src/lib/redis.js` for XLEN, etc.).

`table.released` per-reason soak monitor — □ OPEN (defer until prod has traffic)

Background: Chunk 3 of the table-fixes sweep added a per-reason `table.released` counter to `/api/v1/admin/health/tables` (reasons: `disconnect`, `leave`, `game-end`, `gc-stale`, `gc-idle`, `admin`, `guest-cleanup`, plus `OTHER` catch-all). The shape of the per-reason histogram is the V1-acceptance success metric for "where do tables actually die" — does the disconnect bucket dominate (Safari hang regression) vs. the game-end bucket (healthy completion), is the `OTHER` bucket nonzero (typo'd reason at a call site), is `gc-idle` climbing (idle abandonment runaway), etc.

Why deferred: On staging the only traffic is manual QA — the per-reason distribution reflects the tester's clicks, not real user behaviour. Running a soak there would just measure the test, not the system. The metric's value scales with traffic.

What to do post-prod:

- Schedule a periodic poll of `/api/v1/admin/health/tables` (e.g. hourly via the same scheduler used for tournament sweeps, or a cron-driven Slack/Linear post). Diff against the previous reading and post the per-reason deltas.
- Alert thresholds (rough first cuts; tune from data):
  - `OTHER > 0` for any window → call-site typo, page on-call.
  - `disconnect / game-end > 0.5` over a 1-h window → Safari/network regression suspected; page on-call.
  - `gc-idle` rising > 5/hour while active sessions are non-zero → idle threshold misconfigured.
- Cross-reference with `tableCreateErrors.P2002` (should be ~0 post-chunk-1) and `gc.secondsSinceLastSuccess` (should be < 600s).

Effort: ~2 hours. Reuse the existing scheduler + a small `lib/healthDiff.js` to compute deltas. Pairs naturally with the rest of the Tier 2/3 instrumentation work (see entry above).

Redis-backed `sseSessions` registry — □ OPEN (defer until non-Fly hosting is on the table)

Background: SSE sessions are stored in a per-process `Map` in `backend/src/realtime/sseSessions.js`. With more than one backend machine, follow-up POSTs round-robin via the LB and ~50% land on the machine that doesn't have the session, returning 409 `SSE_SESSION_EXPIRED`. Surfaced on prod 2026-05-04 when prod scaled past 1 backend machine — staging was unaffected because it runs a single machine.

What we shipped instead (2026-05-04): `backend/src/realtime/flyReplay.js` — session ids are minted as `<FLY_MACHINE_ID>.<nanoid>` and `requireSseSession` emits a `Fly-Replay: instance=<owner>` header when a POST lands on a non-owning machine. Fly's edge proxy transparently replays the request on the right machine. Off-Fly (local dev / tests), `FLY_MACHINE_ID` is unset and the path is dormant — sessions are bare nanoids and behavior matches the original sync API.

Why Fly-Replay was the right call now: the actual `res` writable for an open SSE connection lives in one machine's process memory and cannot be migrated. Cross-machine *event delivery* already works today via the redis-streams broker (each machine subscribes and pushes to its own connected clients). The only thing that needs cross-machine coordination is the *session-liveness lookup* — and Fly-Replay routes that lookup back to the connection-owning machine in ~10ms with zero state migration. A Redis-backed registry would solve the same lookup with ~500 LOC of changes (async API ripples through 12 test files + 24 callsites). Until we have a concrete reason to leave Fly, the smaller fix is correct.

When to do the Redis migration:

Trigger on any of:

1. **Non-Fly hosting decision** — moving any backend instance to a non-Fly target (AWS/GCP/Cloudflare/self-hosted K8s) where `Fly-Replay` doesn't exist. Prerequisite for the move, not an after-the-fact fix.
2. **Multi-cloud / multi-provider deployment** — running backend simultaneously on Fly + another platform for redundancy or geo. Fly-Replay only routes within Fly, so cross-provider sessions need a portable lookup layer.
3. **Fly deprecates or rate-limits Fly-Replay** — vendor risk; if the header behavior changes, we need an exit.
4. **Replay-tax becomes a measurable problem** — if backend p95 baselines show the ~10-20ms replay penalty is dominating a hot endpoint and we want to eliminate it, redis lookup on every machine removes the round-trip. Unlikely to matter at our scale, but worth re-checking annually.

What to build (when triggered):

- Move `_sessions` Map to a Redis hash `sse:session:<id>` with 60s TTL, refreshed every `touch()`.
- Move `_byUser` Map to a Redis set `sse:byuser:<userId>`, expired with the parent.
- Keep `_pendingDispose` timers and `_onDispose` callbacks in-memory on the originating machine (they fire off the connection-close event, which always happens locally).
- Make `get`, `forUser`, `joinTable`, `leaveTable`, `touch`, `tablesFor`, `pongRoomsFor` async.
- Add a Lua script (or pipelined commands) for the read-modify-write of `joinedTables` to avoid races between concurrent POSTs on different machines.
- Tear down `flyReplay.js` and revert `events.js` / `realtime.js` edits — Fly-Replay becomes dead code at that point.

Effort estimate: 2-3 days for the rewrite + test updates, plus 1 day of staging soak before promote. Pair it with the move to whichever new hosting target triggers it — the work is mostly the same.

Doc cross-refs: `backend/src/realtime/flyReplay.js` (current implementation), `doc/Realtime_Channels.md` (channel namespace + POST routes affected).